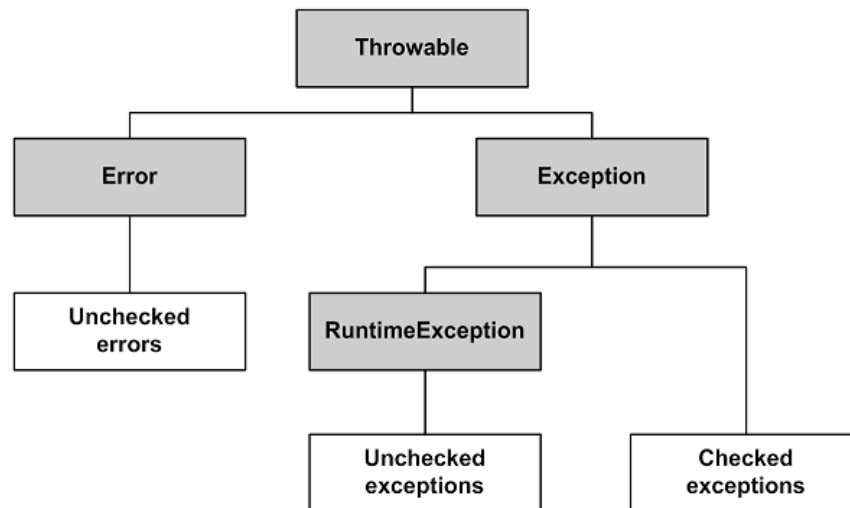


Exception Handling

An exception is an indication of a problem that occurs during a program's execution. Exception handling enables programmers to create applications that can resolve exceptions. All exception types are subclasses of the built-in class `Throwable`. When an exceptional condition arises, an object representing that exception is created and **thrown** in the method that caused the error.



Advantage of Exception Handling

- Allows you to fix the error
- Prevents the program from automatically terminating

Types of Exception

There are three types of exceptions:

Checked Exception: the exceptions that are checked at compile time. In Java, exceptions under **Error** and **RuntimeException** classes are unchecked exceptions, everything else under `Throwable` is checked.

Keywords for exception handling

- **try** block contains program statements that are to be monitored for exceptions
- **catch** block contains statements that catch this exception and handle it in some rational manner
- **throw** is used to manually throw an exception. It is used to throw an exception explicitly.
- **throws** clause is used to specify any exception that is thrown out of a method
- **finally** block contains code that absolutely must be executed after a try block completes

Example,

Without Exception Handling

```
1 package exceptionhandling;
2
3 public class Example1 {
4     public static void main(String args[]) {
5         int d=0, a = 0;
6         a = 42 / d;
7         System.out.println("a = " + a);
8     }
9 }
```

With Exception Handling

```
1 package exceptionhandling;
2
3 public class Example1 {
4     public static void main(String args[]) {
5         int d, a = 0;
6         try { // Monitor a block of code.
7             d = 0;
8             a = 42 / d;
9             System.out.println("This will not be printed.");
10        } catch (ArithmeticException e) { // catch divide-by-zero error
11            System.out.println("Division by zero.");
12        }
13        System.out.println("a = " + a); //After catch statement
14    }
15 }
```

Multiple catch Clauses (Example)

```
1 package exceptionhandling;
2 public class Example2 {
3     public static void main(String args[]) {
4         try {
5             int a = args.length;
6             System.out.println("a = " + a);
7             int b = 42 / a;
8             int c[] = { 1 };
9             c[42] = 99;
10        }
11        catch(ArithmeticException e)
12        { System.out.println("Divide by 0: " + e); }
13        catch(ArrayIndexOutOfBoundsException e)
14        { System.out.println("Array index oob: " + e); }
15        System.out.println("After try/catch blocks."); }
16 }
```

Java's Built-in Exceptions

- **ArithmeticException:** Arithmetic error, such as divide-by-zero.
- **ArrayIndexOutOfBoundsException:** Array index is out-of-bounds.
- **NullPointerException:** Invalid use of a null reference.
- **NumberFormatException:** Invalid conversion of a string to a numeric format.
- **FileNotFoundException:** This Exception is raised when a file is not accessible or does not open.
- **IOException:** It is thrown when an input-output operation failed or interrupted

Example,

```

1 package exceptionhandling;
2 import java.io.File;
3 import java.io.FileNotFoundException;
4 import java.io.FileReader;
5 class FileNotFoundExceptionDemo {
6
7     public static void main(String args[])
8     {
9         try {
10
11             // Following file does not exist
12             File file = new File("E:// file.txt");
13
14             FileReader myFile = new FileReader(file);
15         }
16         catch (FileNotFoundException e) {
17             System.out.println("File does not exist");
18         }
19     }
20 }

```

User-Define Exception

```

1 package exceptionhandling;
2 public class InvalidAgeException extends Exception{
3     InvalidAgeException (String S)
4     {
5         super(S);
6     }
7
8     static void validate (int age) throws InvalidAgeException
9     {
10         if (age < 18)
11             throw new InvalidAgeException ("your age is not valid");
12         else
13             System.out.print("Welcome");
14     }
15
16     public static void main(String[] args)
17     {
18         try {
19             validate(15);
20         }
21         catch (Exception e)
22         {
23             System.out.print(e);
24         }
25     }
26 }

```

Homework

- nested try
- finally block